

ComicCraft – AI Comic Story Creator using Gemini Models

SUB TOPIC `route.py` Development

Activity 3.1: Writing the Main Application Logic in `routes.py`

- Technology: Python
- Framework: Flask
- AI Model: Google Gemini
- Purpose: Generate creative comic stories using AI
- Module: Main application routing and logic

Slide 2 — Role of `routes.py`

Main Application Logic

- `routes.py` manages the communication between the user interface and backend.
- Defines Flask routes for different application functions.
- Receives user inputs such as:
 - Character details
 - Story theme
 - Setting
 - Number of panels
- Sends prompts to the Gemini model.
- Processes the AI-generated response.
- Returns the generated comic story to the webpage.

Basic Flow:

User Input → Flask Route → Gemini API → AI Response → Comic Story Display

Slide 3 — Python Route Implementation

Example: Main Story Generation Route

```
from flask import Blueprint, render_template, request
import google.generativeai as genai

main = Blueprint("main", __name__)

genai.configure(api_key="YOUR_API_KEY")
model = genai.GenerativeModel("gemini-2.0-flash")

@main.route("/")
def home():
    return render_template("index.html")

@main.route("/generate", methods=["POST"])
def generate_story():
    theme = request.form.get("theme")
    characters = request.form.get("characters")

    prompt = f"""
    Create a short comic story.
    Theme: {theme}
    Characters: {characters}

    Divide the story into comic panels
    with dialogue and scene descriptions.
    """

    response = model.generate_content(prompt)

    return render_template(
        "result.html",
        story=response.text
    )
```

Slide 4 — Working of the Application Logic

Step-by-Step Processing

1. Receive Input

- Flask receives the user's comic requirements through a POST request.

2. Create Prompt

- Python combines the user inputs into a structured Gemini prompt.

3. Call Gemini

- The application sends the prompt to the Gemini generative model.

4. Process Response

- The generated text is retrieved from the Gemini response.

5. Display Result

- The generated comic story is passed to `result.html`.

Key Python Components:

- Flask Blueprint
- `@route()` decorators
- `request.form`
- Gemini `generate_content()`
- `render_template()`

Slide 5 — Milestone 3 Outcome

Activity 3.1: Completed Main Application Logic

- Developed the main Flask routes in `routes.py`.
- Connected frontend inputs with backend Python logic.
- Integrated Gemini for AI-based story generation.
- Created structured prompts for comic creation.
- Returned generated stories to the result page.
- Established the core workflow of **ComicCraft**.

Final Workflow:

```
User
↓
ComicCraft Interface
↓
routes.py
↓
Prompt Creation
↓
Gemini Model
↓
Generated Comic Story
↓
Result Page
```

Outcome:

The main backend routing and AI story-generation logic for ComicCraft is implemented.